

Powered Autonomous Delivery Aircraft (PADA)

Integrated Technical Report

Lycans AeroDesign Team

September 13, 2026

Contents

1	Overview & System Constraints Integration	3
2	Baseline Design (Initial Configuration)	3
3	Optimization & Trade-Off Analysis	3
4	Optimized Final Design (Current Configuration)	4
5	Detailed Power Budget and Flight Endurance Calculations	4
5.1	Total Battery Energy Capacity (E_{batt})	4
5.2	Maximum Continuous Current and Discharge Safety Check	5
5.3	Average System Current Draw ($I_{\text{total, avg}}$)	5
5.4	Estimated Flight Endurance (T_{flight})	5
6	Wiring Topology, Exact Port Mapping, and Connectors	5
7	Summary of Optimization Impact	6
8	Master Avionics Wiring Schematic	6
9	Detailed Design	7
9.1	Autonomous System and Software	7
9.1.1	Node and Topic Documentation	7
9.1.2	Camera Source Fallback Chain	7
9.1.3	Reducing Video Buffer Latency	7
9.1.4	Color Space Selection	8
9.1.5	Color Selection	8
9.1.6	Minimum Area Filtering	8
9.1.7	Proximity Filtering for the Obstacle	8
9.1.8	Debounce Counter State Logic	9
9.1.9	Combined Shape and Geometric Verification	9
9.1.10	Steering Zone Boundaries	9

9.1.11	Publish-on-Change Communication	10
9.1.12	Computational Efficiency	10
9.2	Demonstration Evidence and Visual Logs	10
10	Bonus Task 2 (YOLO Training)	10
10.0.1	Model Training and Metrics	10
10.0.2	Node and Topic Documentation	11
10.0.3	Reused Camera and Reliability Patterns	11
10.0.4	Grid Partitioning and Flight Command Logic	11
10.0.5	Control Node Implementation	11
10.0.6	Demonstration Evidence	12
10.0.7	Control Node Implementation	12
11	ROS 2 Architecture, MAVROS & SITL Integration	12
11.1	ROS 2 Architecture	12
11.2	MAVROS and SITL Integration	13
11.3	Command Integration	13
11.4	Testing and Limitations	15

1 Overview & System Constraints Integration

The primary objective of the Avionics and Power Subsystem is to fulfill the Powered Autonomous Delivery Aircraft (PADA) operational requirements within strict physical limits. Based on the aerodynamic constraint analysis:

- **Maximum Gross Weight ($W_{\max}$):** 19.5 oz $\approx$ 552.8 g (5.421 N).
- **Wing Area (S):** 0.08 m².
- **Baseline Wing Loading (W/S):** 67.76 N/m².

From the propulsion dynamic equations, the minimum takeoff Thrust-to-Weight ratio (T/W)_{TO} is established at 0.38. Applying a 20% safety margin for motor/propeller efficiency losses yields a design target of static $(T/W) \geq 0.50$, corresponding to a minimum static thrust requirement of $T_{\text{req}} \geq 276.4$ g.

2 Baseline Design (Initial Configuration)

Initially, a baseline avionics configuration was evaluated using standard off-the-shelf components. Table 1 summarizes the initial electrical inventory including the safety switch, active buzzer, payload camera, and airspeed sensor.

Table 1: Initial (Baseline) Electrical Components Inventory

Component	Power / Rating	Price	Weight	Selected Model / Notes
Battery	3S / 1300mAh (14.43Wh)	1,000 EGP	115 g	LiPo 3S 1300mAh 45C (< 100 Wh)
ESC	30A Cont. (40A Burst)	600 EGP	28 g	30A Brushless ESC with 5V/3A BEC
Motor	180W / Max 18A	900 EGP	55 g	Sunnysky X2212 KV1400
Flight Controller	5V / 500mA	2,250 EGP	38 g	Pixhawk 4 Mini / Matek H743
Receiver	5V / 100mA	750 EGP	8 g	900MHz Receiver (ExpressLRS)
GPS	5V / 150mA	900 EGP	22 g	Holybro Micro M8N GPS Module
Telemetry	500mW / 5V	1,100 EGP	18 g	Telemetry Transceiver
Servos (x5)	1.8 kg-cm Torque	750 EGP	45 g	SG90 Micro Servos (1-2 kg · cm)
Airspeed Sensor	5V / 25mA	850 EGP	10 g	Digital Pitot Tube Sensor Kit
FPV / Payload Cam	5V / 200mA	650 EGP	12 g	Micro FPV / Delivery Camera
Buzzer & E-Switch	5V / Active Piezo	150 EGP	8 g	Active Buzzer & Safety Arming Switch
Total	–	9,900 EGP	359 g	Leaves 193.8g for Airframe

3 Optimization & Trade-Off Analysis

While the initial configuration met all electrical and thrust demands, its cumulative mass of **359 g** consumed too much of the maximum gross weight limit (552.8 g). To resolve this, an avionics optimization trade study was conducted:

1. **Battery Downsizing (1300 mAh $\rightarrow$ 850 mAh):** Saving **40 g** while staying strictly under the < 100 Wh rule.

2. **Motor Sizing Adjustment** (2212 $\rightarrow$ 2204): Saving **27 g** while preserving $(T/W) \approx 0.76$.
3. **ESC Optimization** (30A $\rightarrow$ 20A): Saving **16 g** with a 42% safety margin.
4. **Flight Controller Integration** (Matek H743-WING): Saving **8 g** and integrating internal power distribution and current sensing.

4 Optimized Final Design (Current Configuration)

Table 2 presents the final electrical component inventory following the trade study reductions.

Table 2: Optimized (Final) Electrical Components Inventory

Component	Power / Rating	Price	Weight	Selected Model / Rationale
Battery	3S/850mAh (9.44Wh)	800 EGP	75 g	Tattu LiPo 3S 850mAh 45C (< 100 Wh rule)
ESC	20A Cont. / 25A Burst	450 EGP	12 g	BLHeli_S 20A ESC with 5V BEC
Motor	KV2300 / Max 14A	650 EGP	28 g	SunnySky X2204 KV2300 (Yields $T/W = 0.76$)
Flight Controller	5V / 500mA	2,250 EGP	30 g	Matek H743-Wing (ArduPilot/MAVROS)
Receiver	5V / 100mA	750 EGP	8 g	ExpressLRS 900MHz Receiver (JST-GH)
GPS	5V / 150mA	900 EGP	20 g	Holybro Micro M8N GPS
Telemetry Radio	500mW / 5V	1,100 EGP	18 g	Holybro Sik Telemetry Radio
Servos (x5)	1.8 kg-cm Torque	750 EGP	45 g	SG90 Micro Servos (1–2 kg · cm)
Airspeed Sensor	5V / 25mA	850 EGP	10 g	MS4525DO Digital Pitot Sensor
FPV / Payload Cam	5V / 200mA	650 EGP	12 g	Caddx Ant / Micro FPV Camera
Buzzer & E-Switch	5V / Active Piezo	150 EGP	8 g	Matek Active Buzzer & Safety Arming Switch
Total	–	9,300 EGP	266 g	Leaves 286.8g for Airframe

5 Detailed Power Budget and Flight Endurance Calculations

To mathematically prove the electrical feasibility and endurance of the optimized PADA system, the power and energy equations are structured as follows:

5.1 Total Battery Energy Capacity (E_{batt})

Using the nominal voltage ($V_{\text{nom}} = 11.1 \text{ V}$) and capacity ($C = 0.85 \text{ Ah}$):

$$E_{\text{batt}} = V_{\text{nom}} \times C = 11.1 \text{ V} \times 0.85 \text{ Ah} = 9.435 \text{ Wh}$$

This satisfies the project constraint of being strictly below 100 Wh and avoids high-voltage LiHV chemistries.

5.2 Maximum Continuous Current and Discharge Safety Check

The battery discharge rate is 45C with an 850 mAh (0.85 A) rating:

$$I_{\text{max, discharge}} = C_{\text{rate}} \times \text{Capacity} = 45 \times 0.85 \text{ A} = 38.25 \text{ A}$$

Since the maximum system current draw under full throttle does not exceed 14 A (motor peak 14 A + avionics baseline ~ 0.8 A), the battery operates well within its safe discharge threshold ($14 \text{ A} < 38.25 \text{ A}$).

5.3 Average System Current Draw ($I_{\text{total, avg}}$)

During normal cruise conditions, the motor draws an average of 5.5 A, while the avionics suite draws an aggregate baseline current of ≈ 0.8 A:

$$I_{\text{total, avg}} = I_{\text{motor, cruise}} + I_{\text{avionics, baseline}} = 5.5 \text{ A} + 0.8 \text{ A} = 6.3 \text{ A}$$

5.4 Estimated Flight Endurance (T_{flight})

Applying an 80% Depth of Discharge (DOD) factor to preserve battery longevity:

$$T_{\text{flight}} = \left(\frac{\text{Capacity} \times \text{DOD}}{I_{\text{total, avg}}} \right) \times 60 = \left(\frac{0.85 \text{ Ah} \times 0.80}{6.3 \text{ A}} \right) \times 60 \approx 6.47 \text{ minutes}$$

This provides a sufficient operational window for autonomous mission execution, waypoint navigation, obstacle avoidance, and payload deployment.

6 Wiring Topology, Exact Port Mapping, and Connectors

Table 3 outlines the precise physical connections, wire types, gauges (AWG), and Matek H743-WING port mapping matching the schematics architecture cleanly within the page limits.

Table 3: Matek H743-WING Port Mapping and Wiring Specifications

Subsystem	Device	Controller	Port	Wire Gauge	Type	&	Connection Notes / Safety
Battery PDB/ESC	to	Battery	Pads (+BAT/ - BAT)	14 AWG Wire	Silicone		Main power input with XT60 & Cap (470 μ F/35V)
Brushless Motor		ESC Motor	Outputs (U, V, W)	16 AWG Wire	Silicone		Secure bullet connectors soldered to ESC
ESC Signal		ESC / S1	PWM Pad + GND + 5V	26/24 AWG			Ground, 5V BEC, and S1 PWM signal
GPS Module		UART2	(TX2/RX2/5V/GND)	28 AWG 4-pin	JST-GH		Connected via JST-GH 4Pins to UART2
Compass (Mag)		I2C1	(CL1/DA1/5V/GND)	28 AWG 4-pin	JST-GH		Connected via JST-GH 4Pins to I2C1 bus
900MHz Receiver		UART1	(TX1/RX1/5V/GND)	28 AWG 4-pin	JST-GH		Connected via JST-GH 4Pins to UART1
Telemetry Radio		UART7	(TX7/RX7/5V/GND)	28 AWG 4-pin	JST-GH		Connected via JST-GH 4Pins to UART7
Servo 1 (Left Aileron)		PWM Output Pad	S2	24 AWG Wire	Silicone		Direct PWM control signal
Servo 2 (Right Aileron)		PWM Output Pad	S3	24 AWG Wire	Silicone		Direct PWM control signal
Servo 3 (Elevator)		PWM Output Pad	S4	24 AWG Wire	Silicone		Direct PWM control signal
Servo 4 (Rudder)		PWM Output Pad	S5	24 AWG Wire	Silicone		Direct PWM control signal
Servo 5 (Flaps)		PWM Output Pad	S6	24 AWG Wire	Silicone		Direct PWM control signal
Payload Camera		Video IN	(VIN) + 5V/GND	28 AWG Coaxial	Shielded		Real-time FPV feed and visual target
Buzzer & E-Switch		BUZ, GND, LED, Safety Pads		28 AWG Connectors	JST		Mandatory safety switch and buzzer

7 Summary of Optimization Impact

Through this engineering optimization, the total avionics mass was reduced from **359 g to 266 g** (a **25.9% mass reduction**), saving **93 g** in total. This increases the allowable airframe structural mass allocation to **286.8 g**, ensuring sufficient structural integrity without exceeding the 552.8 g maximum gross weight limit.

8 Master Avionics Wiring Schematic

Figure ?? displays the integrated electrical pinout, power bus routing, and sensor bus topology for the PADA UAV system using the Matek H743-WING controller matching the exact routing layout.

9 Detailed Design

9.1 Autonomous System and Software

This section explains the design choices I made while building my perception node, and why I wrote the code the way I did.

9.1.1 Node and Topic Documentation

My perception pipeline is implemented as a single ROS 2 node, `perception_publisher_node`, which subscribes to no topics and publishes on two:

- `flight_movement_command` (`std_msgs/String`): carries obstacle-avoidance commands – "MOVE LEFT", "MOVE RIGHT", "MOVE DOWN", or "CONTINUE".
- `flight_mode_command` (`std_msgs/String`): carries flight-mode commands triggered by visual target recognition.

I chose `std_msgs/String` over a custom message type because the command vocabulary is small and fixed. A custom message would only be worth the added complexity if I needed to send extra structured data alongside each command, such as an estimated distance or confidence score, which wasn't necessary at this stage.

9.1.2 Camera Source Fallback Chain

I didn't want this code to only work on my own machine with my own phone connected. The camera setup tries my phone's live RTSP stream first, and falls back to a normal local webcam if that stream is unavailable. This makes the node testable by anyone with basic camera hardware, without requiring my specific network setup, while the live phone stream remains the actual source used during our real demonstration.

Listing 1: Camera fallback: phone stream first, local webcam second.

```
1 self.cap = cv2.VideoCapture(src, cv2.CAP_FFMPEG)
2 self.cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)
3
4 if not self.cap.isOpened():
5     self.get_logger().warn(f"Phone stream {CAMERA_SOURCE}
6         unavailable. Trying default webcam (0)...")
7     self.cap = cv2.VideoCapture(0)
```

9.1.3 Reducing Video Buffer Latency

When streaming video over a network, frames can build up in the background and create a noticeable delay. To make sure the aircraft reacts to what the camera sees right now instead of a few seconds ago, I set OpenCV's buffer size to 1. In the main loop, I grab the latest frame using a 4-frame skip loop so fresh images are processed at 15 Hz. If a frame fails to load, a warning is logged and the function exits early to keep the node from crashing.

Listing 2: Grabbing the video frame and dropping buffer lag in perception_publisher_node.py.

```
1 for _ in range(4):
2     if not self.cap.grab():
3         break
4 ret, frame = self.cap.retrieve()
5 if not ret:
6     self.get_logger().warn("Failed to grab frame from video
7                             source", throttle_duration_sec=3.0)
8     return
```

9.1.4 Color Space Selection

I converted every incoming BGR video frame to the HSV color space before doing any detection. RGB is tricky because a color's brightness is mixed into all three channels, making an orange color look completely different under bright sun versus a shadow. HSV separates color (hue) from brightness, so my thresholds stay stable under changing lighting conditions.

9.1.5 Color Selection

Three distinct colors were picked that are far apart on the HSV hue wheel: orange for obstacles, green for the LOITER mode target, and blue for the MANUAL mode target. Spacing out the colors prevents the system from confusing an obstacle for a flight command. (LOITER and MANUAL modes were picked according to my teammate's preference for the target-to-mode mapping.)

9.1.6 Minimum Area Filtering

Cameras usually pick up small visual noise or stray background reflections. To ignore small blobs, I required any detected shape to cover a minimum portion of the screen before treating it as a real detection (0.2% of total frame area for obstacles, and 0.3% for command targets).

9.1.7 Proximity Filtering for the Obstacle

Blob size alone isn't enough to confirm if an obstacle is close, so I added a height check: the bounding box height must be at least 15% of the total frame height. This acts as a simple proximity check so the system only reacts when an obstacle is close enough to be in the way.

Listing 3: Filtering out small noise and verifying obstacle proximity height.

```
1 if cv2.contourArea(largest) > (h * w * 0.002):
2     x, y, bw, bh = cv2.boundingRect(largest)
3     if (bh / h) >= PROXIMITY_HEIGHT_FRAC:
4         seen_this_frame = True
```

9.1.8 Debounce Counter State Logic

To stop the aircraft from jittering when a frame drops or flickers, I used detection counters. Two positive frames in a row are required to confirm a hazard (`CONFIRM_NEEDED = 2`), but three clean frames in a row are required to clear it (`CLEAR_NEEDED = 3`). Making it harder to clear a hazard than to confirm one is a deliberate safety choice: it's safer to stay in an avoidance state a moment longer than to stop dodging too early.

Listing 4: Asymmetric debounce counters to confirm and clear hazards.

```
1 if seen_this_frame:
2     self.hazard_seen_count += 1
3     self.hazard_miss_count = 0
4 else:
5     self.hazard_miss_count += 1
6     self.hazard_seen_count = 0
7
8 if self.hazard_seen_count >= CONFIRM_NEEDED:
9     self.hazard_confirmed = True
10 if self.hazard_miss_count >= CLEAR_NEEDED:
11     self.hazard_confirmed = False
```

9.1.9 Combined Shape and Geometric Verification

To prevent background objects from accidentally triggering a mode change, I check shape geometry on top of color masking. Green targets must be circular (circularity ≥ 0.65) to trigger `LOITER`, while blue targets must be square-ish (aspect ratio between 0.7 and 1.3) to trigger `MANUAL`. Requiring both the right color and the right shape avoids false triggers from a similarly-colored object that doesn't share the correct geometry.

Listing 5: Shape verification for Green Circle (`LOITER`) and Blue Square (`MANUAL`).

```
1 circularity = (4 * np.pi * area) / (peri ** 2)
2 aspect_ratio = bw / float(bh)
3
4 if (shape_type == "circle" and circularity >= 0.65) or \
5     (shape_type == "square" and 0.7 <= aspect_ratio <= 1.3):
6     seen_target = True
```

9.1.10 Steering Zone Boundaries

When an obstacle is confirmed, I pick a dodge direction based on where its center sits across the screen width. If it is in the left 40%, the system dodges right. If it is in the right 40%, it dodges left. If it is in the middle 20%, it tells the vehicle to move down instead, since a lateral dodge is not a safe option for an obstacle directly ahead.

Listing 6: Deciding dodge directions using screen position.

```
1 if cx < w * 0.4:
2     obstacle_cmd = "MOVE RIGHT"
3 elif cx > w * 0.6:
4     obstacle_cmd = "MOVE LEFT"
5 else:
```

```
obstacle_cmd = "MOVE DOWN"
```

9.1.11 Publish-on-Change Communication

The node runs 15 times per second, but publishing the exact same message every frame wastes bandwidth and floods the topic with redundant data. I track the last command sent and only publish a new message on `flight_movement_command` or `flight_mode_command` when the output command actually changes.

9.1.12 Computational Efficiency

I measured performance using a standalone test script running the same detection logic against the live phone camera feed, independent of ROS 2, logging frame timing and system resource usage. Over a representative test run:

- **Average frame rate:** approximately 30 FPS, roughly double the 15 Hz processing loop rate used in the deployed ROS 2 node, indicating the detection pipeline itself is not the bottleneck.
- **CPU usage:** 35–39% of a single core on standard laptop hardware.
- **Memory usage:** a stable 135.5 MB throughout the test, with no observed growth over time, indicating no memory leak in the detection loop.

These results confirm the pipeline runs comfortably within real-time constraints, leaving significant headroom for ROS 2 middleware and MAVROS communication running concurrently on the same machine.

9.2 Demonstration Evidence and Visual Logs

To verify detection performance, a screen recording is archived on Google Drive:

- **System Video Demonstration:** [Google Drive – Autonomous System Screen Recording](#)

10 Bonus Task 2 (YOLO Training)

Instead of relying on fixed color masks, I integrated a custom-trained YOLO model to recognize a balloon target under changing light and angles. Color masking only recognizes exactly the colors and shapes it is told to look for; a trained model doesn't need a hand-tuned color range at all.

10.0.1 Model Training and Metrics

I trained a YOLO11n model using Ultralytics on a balloon dataset from Kaggle, over 30 epochs on CPU (about 8.95 hours). On my validation set of 109 images, it scored a precision of 0.937, recall of 0.897, and mAP50 of 0.937. I chose the smallest YOLO11 variant since my target hardware is a CPU-only laptop with no GPU.

Listing 7: Loading the trained model once at startup.

```
self.model = YOLO(MODEL_PATH)
```

10.0.2 Node and Topic Documentation

This pipeline runs as its own node, `perception_yolo_node`, publishing to its own topic, `balloon_tracking_command` (`std_msgs/String`), kept separate from the main pipeline's topics since it is a distinct detection method.

10.0.3 Reused Camera and Reliability Patterns

Rather than building new reliability logic, I reused the same phone-stream-then-webcam fallback and the same seen/miss debounce counters from my main pipeline.

Listing 8: Same debounce pattern as the main pipeline, applied to grid sectors.

```
1 if detected_sector is not None:
2     self.seen_count += 1
3     self.miss_count = 0
4 else:
5     self.miss_count += 1
6     self.seen_count = 0
7
8 if self.seen_count >= CONFIRM_NEEDED:
9     self.confirmed_sector = detected_sector
10 if self.miss_count >= CLEAR_NEEDED:
11     self.confirmed_sector = None
```

10.0.4 Grid Partitioning and Flight Command Logic

The camera view is split into a 3×3 grid. Only the four corner sectors trigger a command – top corners trigger RTL, bottom corners trigger AUTO. Any middle sector produces no command, since its position doesn't clearly indicate a direction.

Listing 9: Triggering flight modes based on grid corner sectors.

```
1 if self.confirmed_sector == 'top_left' or self.confirmed_sector
   == 'top_right':
2     command = 'RTL'
3 elif self.confirmed_sector == 'bottom_left' or self.
   confirmed_sector == 'bottom_right':
4     command = 'AUTO'
```

10.0.5 Control Node Implementation

To process automated flight commands derived from the YOLO perception pipeline, a dedicated ROS 2 subscriber node, `yolo_flight_control_node`, was implemented using `rclpy`. The node subscribes to the `balloon_tracking_command` topic to receive real-time mode transition instructions based on visual balloon sector tracking.

To optimize message processing and avoid redundant state executions, the node tracks the incoming flight mode state and processes updates only when a new transition command is detected.

10.0.6 Demonstration Evidence

A screen recording of the YOLO-based balloon detection running live is archived on Google Drive:

- **YOLO Balloon Detection Demonstration:** [Google Drive – YOLO Balloon Tracking Screen Recording](#)

10.0.7 Control Node Implementation

To process automated flight commands derived from the YOLO perception pipeline, a dedicated ROS 2 subscriber node, `yolo_flight_control_node`, was implemented using `rclpy`. The node subscribes to the `balloon_tracking_command` topic to receive real-time mode transition instructions based on visual balloon sector tracking.

To optimize message processing and avoid redundant state executions, the node tracks the incoming flight mode state and processes updates only when a new transition command is detected.

11 ROS 2 Architecture, MAVROS & SITL Integration

11.1 ROS 2 Architecture

The ROS 2 communication layer connects real-time computer vision processing decisions from the `pada_vision` perception package to the simulated fixed-wing aircraft controller. The primary decision-making control node, `node1`, was developed in Python using `rclpy`. It ingests target classifications and hazard detection data from the perception nodes, monitors aircraft spatial orientation and flight status, and interfaces directly with the flight controller via MAVROS topics and service calls.

The core ROS 2 and MAVROS interfaces integrated into `node1` are outlined in Table 4.

Table 4: Main ROS 2 and MAVROS interfaces utilized by the `node1` control node.

Interface	Type	Purpose
<code>/mavros/state</code>	Topic	Monitors the aircraft operational state and active flight mode.
<code>/mavros/local_position/pose</code>	Topic	Provides real-time local aircraft frame coordinates.
<code>/flight_movement_command</code>	Topic	Receives directional hazard evasion vector commands.
<code>/flight_mode_command</code>	Topic	Receives visual target flight-mode override commands.
<code>/mavros/set_mode</code>	Service	Requests flight-mode transitions from the flight controller.
<code>/mavros/setpoint_position/local</code>	Topic	Publishes relative spatial target coordinates during GUIDED mode.

The topology of the active ROS 2 computational graph was validated using `rqt_graph`. This confirmed proper node connections, topic subscriptions, and service bindings between the `pada_vision` nodes, `node1`, and MAVROS wrappers.

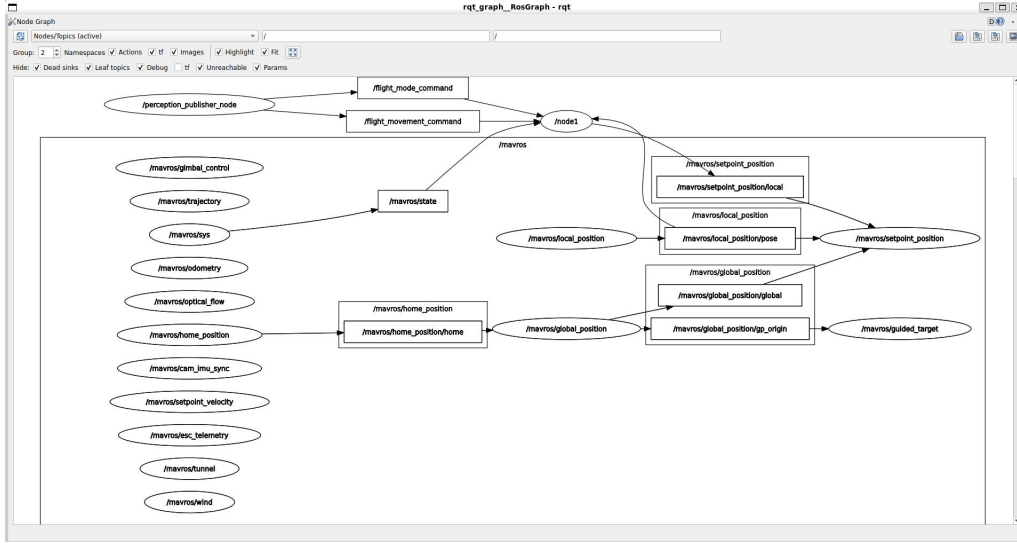


Figure 1: ROS 2 computational graph showing active nodes, publishers, and MAVROS links.

11.2 MAVROS and SITL Integration

MAVROS serves as the bridge between ROS 2 nodes and the ArduPilot Software-in-the-Loop (SITL) fixed-wing flight simulator. By abstracting the low-level MAVLink protocol into ROS 2 messaging structures, the system streams continuous flight status telemetry and issues high-level navigational overrides without requiring raw MAVLink packet construction.

Flight states are continuously validated via `/mavros/state`, while global mode changes are triggered through the `/mavros/set_mode` service. Local frame coordinates (x, y, z) are updated through subscriptions to `/mavros/local_position/pose`.

The end-to-end data pipeline is structured as:

ROS 2 (`pada_vision` / `node1`) → MAVROS → ArduPilot SITL →
QGroundControl

QGroundControl (QGC) was integrated into the simulation environment to provide real-time telemetry verification, mission tracking, and visual confirmation of mode changes initiated by ROS 2 decision events.

11.3 Command Integration

The decision node processes two distinct streams of operational commands from the vision pipeline.

For **hazard detection and spatial evasion**, the system issues relative directional displacement vectors over `/flight_movement_command`:

MOVE LEFT, MOVE RIGHT, MOVE DOWN

Upon receiving an evasion vector, **node1** calculates relative offsets from the aircraft's current local position, publishes local setpoints to `/mavros/setpoint_position/local`, and transitions the flight mode to GUIDED. For example, a MOVE RIGHT instruction increments the local x -axis target by 10 units.

When the perception pipeline flags the hazard zone as clear via the CONTINUE command, the node clears temporary position offsets and requests AUTO mode to resume the pre-programmed waypoints.

For **visual target identification**, the system directly triggers flight-mode transitions. The mapping applied during flight verification is summarized in Table 5.

Table 5: Visual target classification to ArduPilot flight-mode mapping.

Visual Target Class	Triggered Flight Mode
Green circle	LOITER
Blue square	MANUAL

Target classification signals trigger **node1** to execute synchronous calls against the `/mavros/set_mode` service to switch modes on the flight controller.

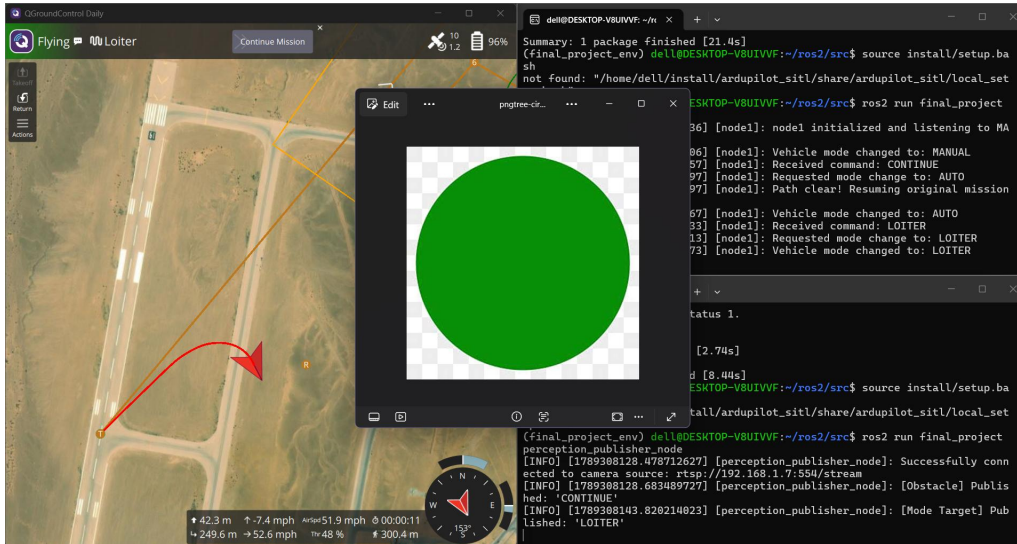


Figure 2: Visual target identification test showing perception output, node log execution, and mode change in QGroundControl.

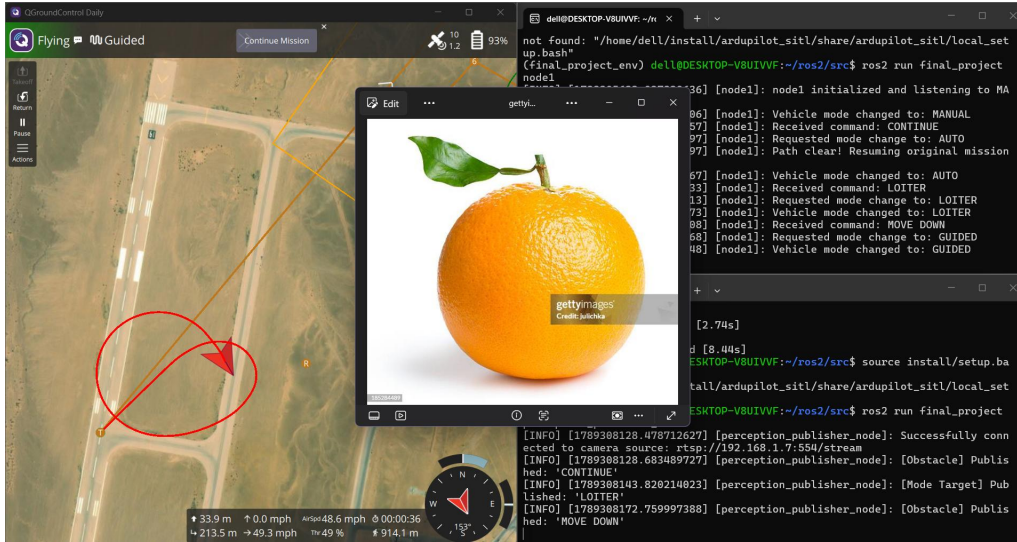


Figure 3: Hazard avoidance test demonstrating movement command reception, setpoint generation, and flight state modification.

11.4 Testing and Limitations

Simulation testing verified end-to-end ROS 2–MAVROS messaging and confirmed reliable execution of automated flight-mode transitions. However, local position setpoint control during GUIDED mode hazard avoidance exhibited unexpected behaviors in SITL testing: although the aircraft successfully switched to GUIDED mode, spatial position updates published to MAVROS were not accepted by the ArduPilot fixed-wing controller as valid trajectory targets.

An alternative rally-point detour strategy was also evaluated. During execution, the aircraft briefly initiated Return-to-Launch (RTL) before reverting to AUTO mode; however, rather than maintaining its active waypoint route, the controller prioritized direct trajectory alignment toward the landing waypoint. This behavior is likely linked to fixed-wing navigation parameters in ArduPilot SITL rather than a defect in the ROS 2 node architecture.

An automated timeout-based recovery strategy was evaluated to handle unacknowledged GUIDED mode states (returning to AUTO after a 3-second hold). This was ultimately omitted from the final node build because the perception pipeline operates on state-change events rather than continuous frame streams; implementing a simple timeout risk incorrectly interrupting valid evasion maneuvers.

Consequently, the current system successfully demonstrates robust ROS 2 software integration, MAVROS message handling, and vision-driven mode control for the PADA system, with local trajectory execution under GUIDED mode slated for future control parameter tuning.